# 0763. Partition Labels

Medium · Greedy

leetcode.com/problems/partition-labels

# Problem

- Split s into as many parts as possible so each letter appears in at most one part. Return the part sizes.

# Intuition

- Record each letter's last index. While scanning, the current part must extend at least to the last occurrence of every letter seen in it.
- When i reaches that end, the part can be closed.

# Brute Force

- Try all cut positions and validate: O(n^2) or worse.

# Optimal Approach

- Last[c] = last index of c.
- Start = 0, end = 0. For i: end = max(end, last[s[i]]); if i == end: record end - start + 1; start = i + 1.

# Worked Example

```
"ababcbacadefegdehijhklij".

part 1: a..a ends at 8   -> "ababcbaca" (9)
part 2: d..e ends at 15  -> "defegde"   (7)
part 3: h..j ends at 23  -> "hijhklij"  (8)
```

# Java Solution

```java
public class PartitionLabels {
    public List<Integer> partitionLabels(String s) {
        int[] last = new int[26];
        for (int i = 0; i < s.length(); i++) last[s.charAt(i) - 'a'] = i;
        List<Integer> res = new ArrayList<>();
        int start = 0, end = 0;
        for (int i = 0; i < s.length(); i++) {
            end = Math.max(end, last[s.charAt(i) - 'a']);
            if (i == end) { res.add(end - start + 1); start = i + 1; }
        }
        return res;
    }
}
```

# Complexity

- Time: O(n). Space: O(26).

# Pitfalls

- End only grows within a part.